



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

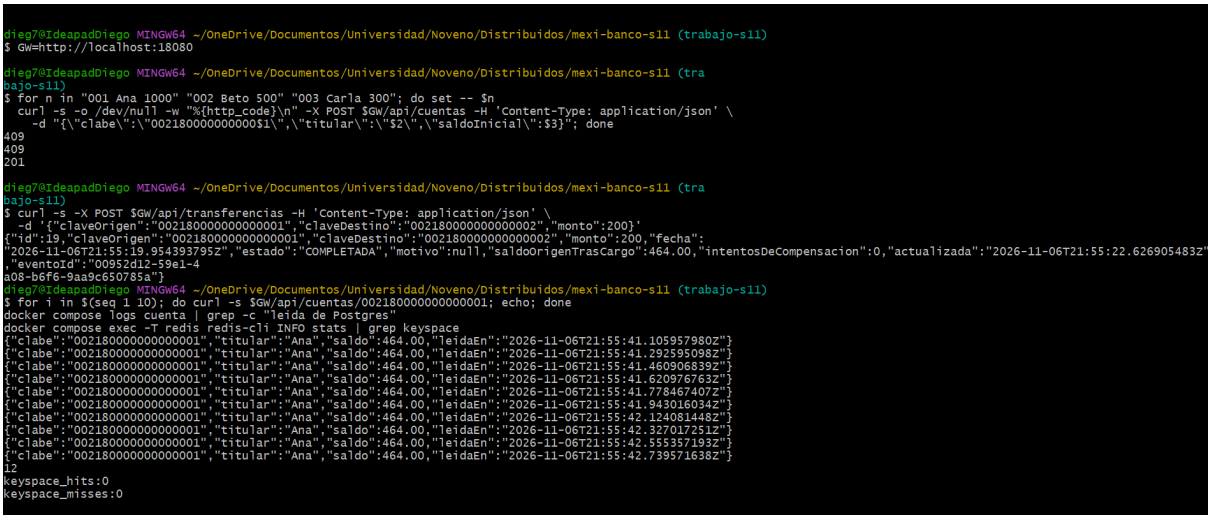
SISTEMAS DISTRIBUIDOS

LABORATORIO SESIÓN 11

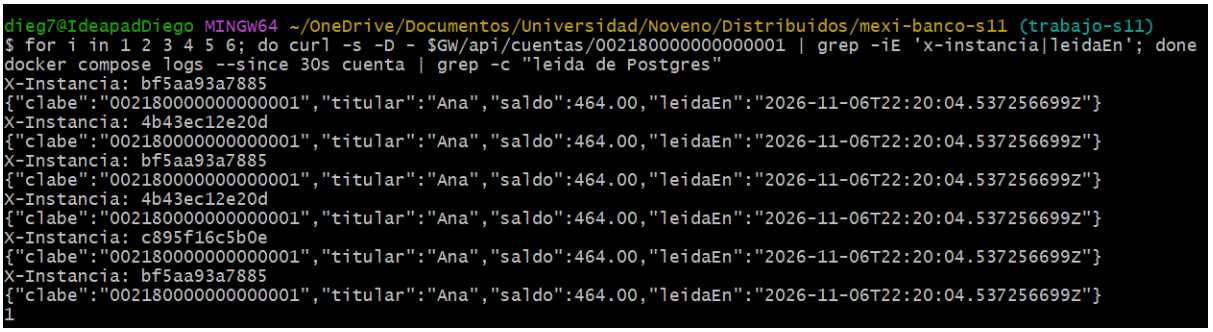
CONTRERAS DEL ÁNGEL DIEGO ADRIÁN

SEMESTRE 2027-1

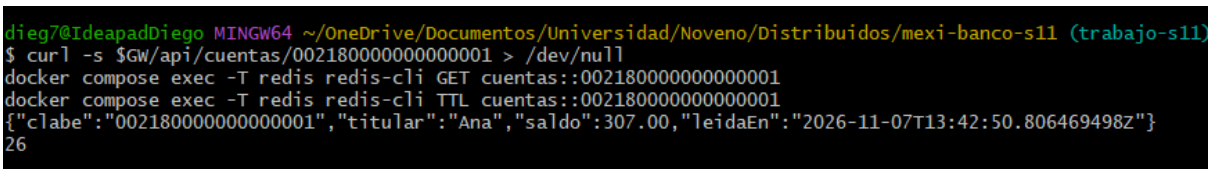
07 DE NOVIEMBRE DE 2026



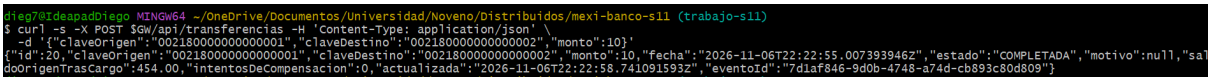
Diez consultas del saldo de Ana. Cada respuesta trae un leidaEn distinto, porque las diez se leyeron de Postgres. El log cuenta 12 lecturas, y Redis marca keyspace_hits:0 y keyspace_misses:0, porque todavía no se usa.

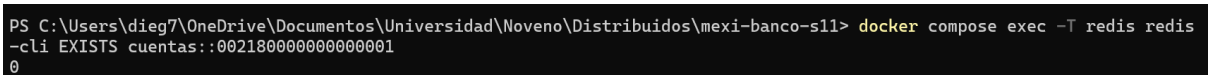


Pedí el saldo de Ana seis veces y me contestaron tres copias distintas del servicio. Aun así, todas muestran la misma hora de lectura, y el registro cuenta una sola consulta a la base de datos. Las tres copias comparten la misma caché.



Después de consultar el saldo, revisé qué guardó Redis: el saldo de Ana con su hora de lectura, y el contador marca 26 segundos. Es el tiempo que le queda antes de que se borre solo.





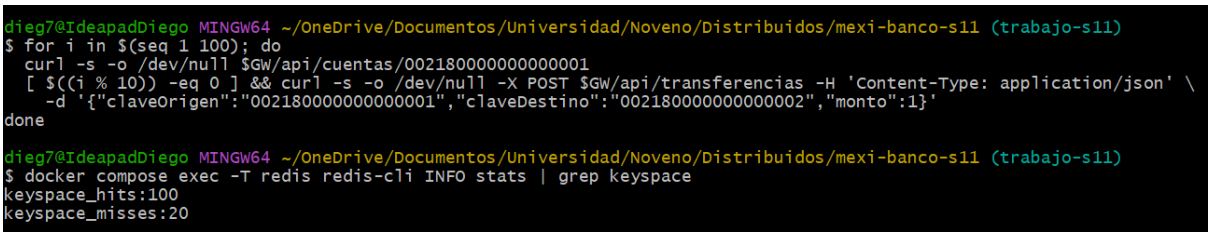
Hice una transferencia de 10 de Ana a Beto, que salió completada. Luego pregunté si el saldo de Ana seguía guardado en Redis y la respuesta fue 0, o sea que ya no está. La transferencia borró la copia para que nadie vea un saldo viejo.



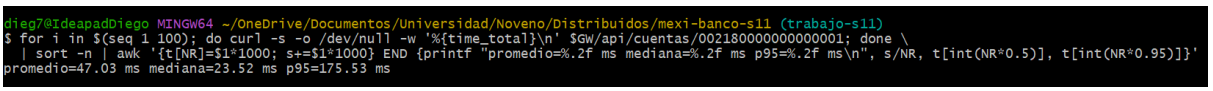
Volví a pedir el saldo de Ana y salió el valor nuevo, 454.00, con una hora de lectura nueva. Como la copia se había borrado, el sistema fue a la base de datos y guardó el saldo actualizado.



Hice cien consultas seguidas del saldo de Ana. La caché acertó 99 veces y falló solo 1, así que el porcentaje de aciertos es de 99 %. Solo la primera consulta tuvo que ir a la base de datos, y las otras 99 se contestaron con el saldo ya guardado.



Repetí las cien consultas, pero con una transferencia de 1 peso cada diez consultas. La caché acertó 100 veces y falló 20, lo que da 83.3 % de aciertos. Baja porque cada transferencia borra el saldo guardado de Ana y de Beto, y la siguiente consulta de cada uno tiene que volver a la base de datos. Con 10 transferencias salen 20 fallos.



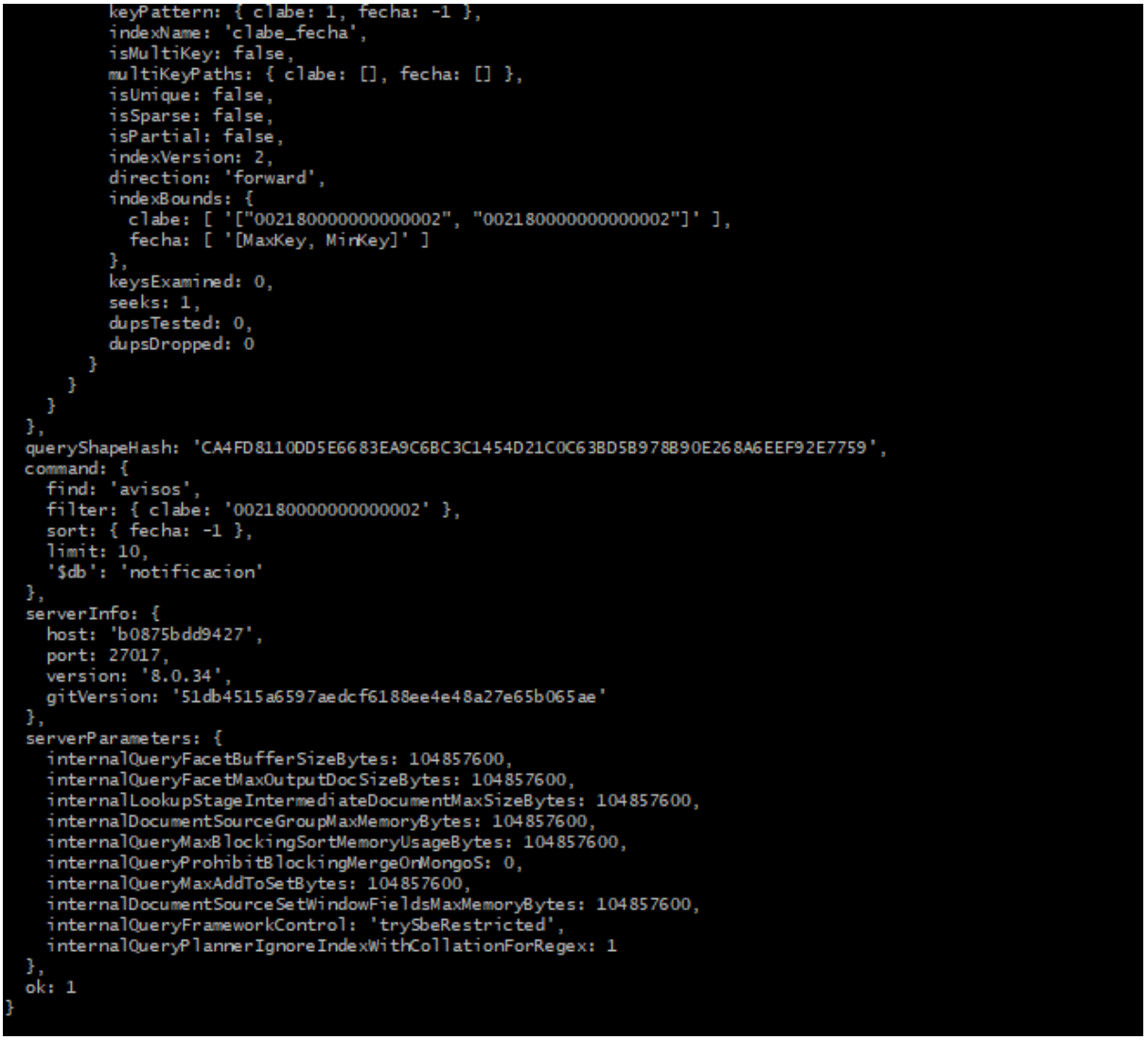
Medí cuánto tarda cada una de las cien consultas con la caché encendida. En esta corrida el promedio fue de 47.03 ms, la mediana de 23.52 ms y el p95 de 175.53 ms. El p95 significa que 95 de cada 100 consultas tardaron ese tiempo o menos.

```
dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco-s11 (trabajo-s11)
$ CACHE_TIPO=none docker compose up -d cuenta
[+] up 6/6
 ✔ Container mexi-banco-v06-db-1         Healthy                                                
```

Hice doce transferencias de 1 peso de Ana a Beto y consulté los avisos de Beto. El sistema devolvió exactamente 10, aunque se hicieron doce. Todos son del canal CORREO, porque Beto pidió que le avisen por correo. Vienen ordenados del más reciente al más viejo: el primero deja a Beto con saldo de 1105.00 y el último con 1096.00.

```
dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.getIndexes()'
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { clabe: 1, fecha: -1 }, name: 'clabe_fecha' }
]

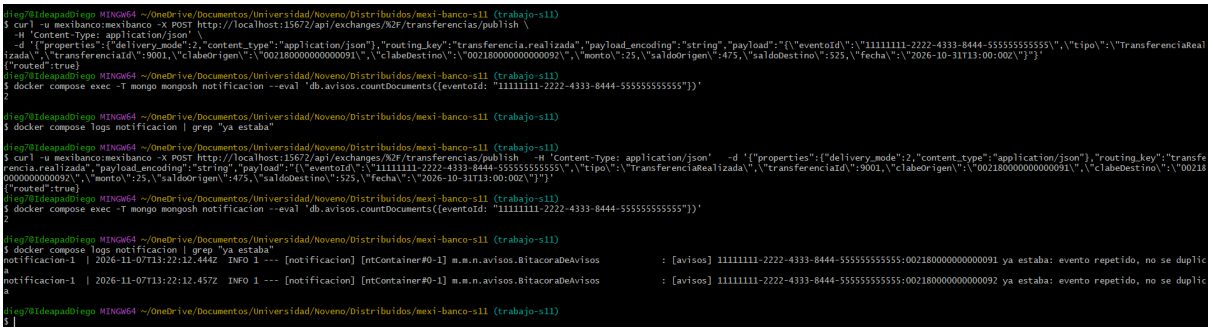
dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe: "002180000000000002"}).sort({fecha: -1}).limit(10).explain("executionStats")'
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'notificacion.avisos',
    parsedQuery: { clabe: { '$eq': '002180000000000002' } },
    indexFilterSet: false,
    queryHash: 'C73271DB',
    planCacheShapeHash: 'C73271DB',
    planCacheKey: '501A071D',
    optimizationTimeMillis: 140,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 10,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { clabe: 1, fecha: -1 },
          indexName: 'clabe_fecha',
          isMultiKey: false,
          multiKeyPaths: { clabe: [], fecha: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            clabe: [ '["002180000000000002", "002180000000000002"]' ],
            fecha: [ '[MaxKey, MinKey]' ]
          }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 0,
    executionTimeMillis: 155,
    totalKeysExamined: 0,
    totalDocsExamined: 0,
    executionStages: {
      isCached: false,
      stage: 'LIMIT',
      nReturned: 0,
      executionTimeMillisEstimate: 11,
      works: 1,
      advanced: 0,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
```



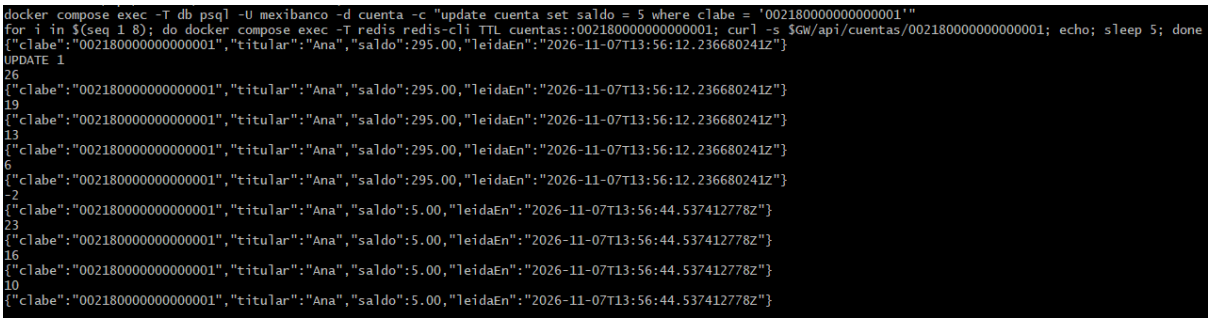
Pedí a Mongo la lista de índices de la colección de avisos. Aparecen dos: el que Mongo trae por defecto (_id_) y el que creé yo, llamado clabe_fecha, que ordena los avisos por cuenta y por fecha. Sirve para encontrar rápido los últimos avisos de una cuenta.

```
dieg7@IdeapadDiego MINGW64 ~/OneDrive/Documentos/Universidad/Noveno/Distribuidos/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe: "002180000000000002"}).sort({fecha: -1}).limit(10).hint({$natural: 1}).explain("executionStats")'
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'notificacion.avisos',
    parsedQuery: { clabe: { '$eq': '002180000000000002' } },
    indexFilterSet: false,
    queryHash: 'C73271DB',
    planCacheShapeHash: 'C73271DB',
    planCacheKey: '501A071D',
    optimizationTimeMillis: 42,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { fecha: -1 },
      memLimit: 104857600,
      limitAmount: 10,
      type: 'simple',
      inputStage: {
        stage: 'COLLSCAN',
        filter: { clabe: { '$eq': '002180000000000002' } },
        direction: 'forward'
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 0,
    executionTimeMillis: 59,
    totalKeysExamined: 0,
    totalDocsExamined: 0,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 0,
      executionTimeMillisEstimate: 6,
      works: 2,
      advanced: 0,
      needTime: 1,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      sortPattern: { fecha: -1 },
      memLimit: 104857600,
      limitAmount: 10,
      type: 'simple',
      totalDataSizeSorted: 0,
      usedDisk: false,
      spills: 0,
      spilledDataStorageSize: 0,
      inputStage: {
        stage: 'COLLSCAN',
        filter: { clabe: { '$eq': '002180000000000002' } },
        nReturned: 0,
        executionTimeMillisEstimate: 0,
        works: 1,
        advanced: 0,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        direction: 'forward',
        docsExamined: 0
      }
    }
  },
  queryShapeHash: 'CA4FD8110DD5E6683EA9C6BC3C1454D21C0C63BD5B978B90E268A6EEF92E7759',
  command: {
    find: 'avisos',
    filter: { clabe: '002180000000000002' },
    sort: { fecha: -1 },
    hint: { '$natural': 1 },
    limit: 10,
    '$db': 'notificacion'
  },
  serverInfo: {
    host: 'b0875bdd9427',
    port: 27017,
    version: '8.0.34',
    gitVersion: '51db4515a6597aedcf6188ee4e48a27e65b065ae'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
```

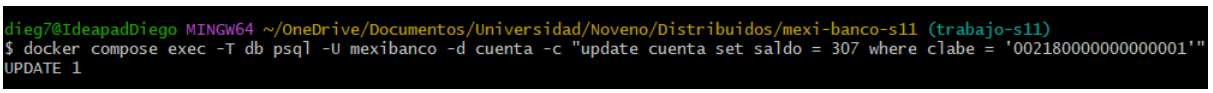
Repetí la búsqueda, pero le pedí a Mongo que no usara el índice. Ahora el plan muestra COLLSCAN, que significa que recorrió la colección completa, aviso por aviso, y después los ordenó. Las estadísticas dicen que revisó [todos los avisos de la colección, por ejemplo 30] para devolver los mismos 10.



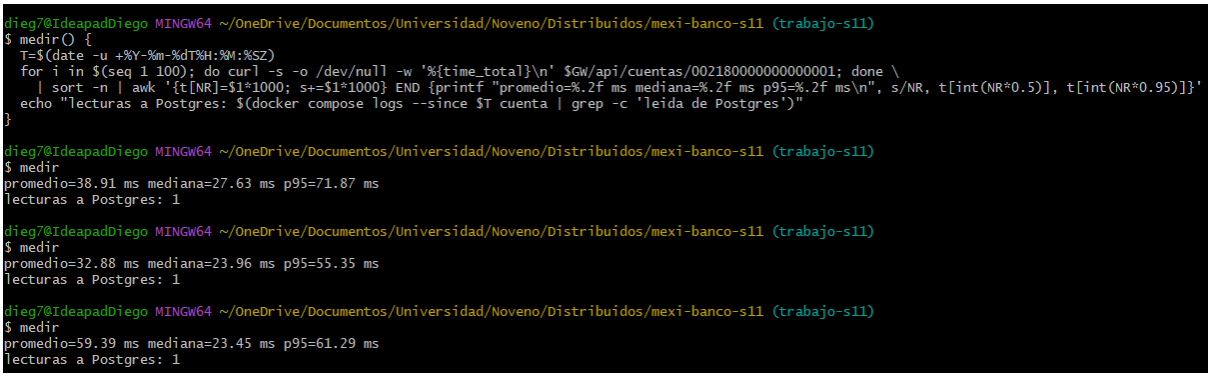
Mandé el mismo evento dos veces. Las dos veces el sistema contestó routed:true, es decir, que lo recibió. Aun así, en Mongo quedaron solo 2 avisos y no 4. Cada aviso tiene un identificador único formado por el evento y la cuenta, así que Mongo rechaza los repetidos.



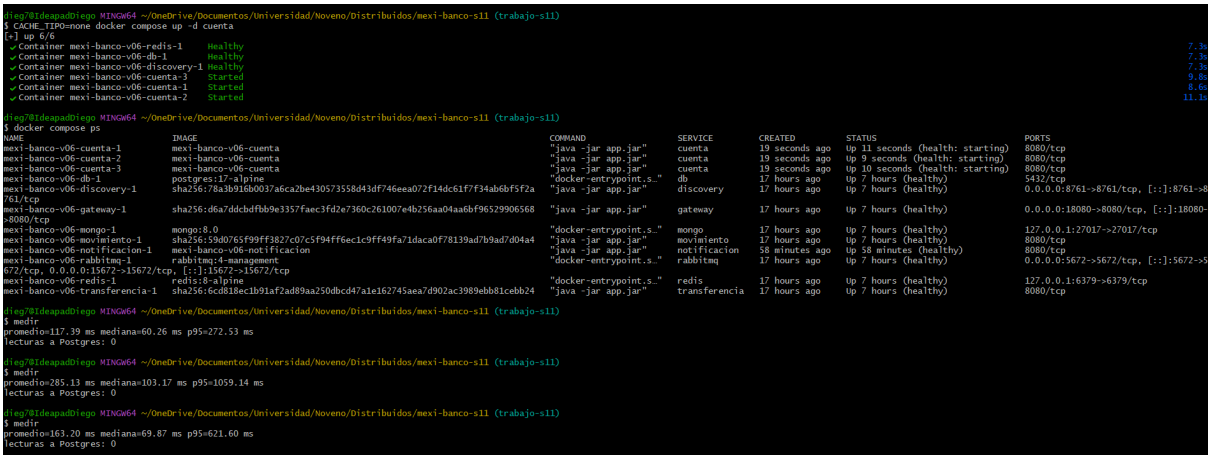
Cambié el saldo de Ana directamente en la base de datos, sin pasar por la aplicación. Durante 26 segundos el cliente siguió viendo el saldo anterior 295, aunque en la base de datos ya había 5. La caché no se enteró del cambio. Cuando el contador llegó a -2, la copia caducó y apareció el saldo real. Así se ve que la caché puede mostrar un dato atrasado.



Después de la prueba deje el saldo como estaba.



Medí cien consultas del saldo de Ana, tres veces, con la caché encendida. Los tiempos fueron para la primera corrida promedio: 38.91 mediana: 27.63 p95: 71.87, para la segunda corrida promedio: 32.88 mediana: 23.96 p95: 55.35 y para la tercera corrida promedio: 59.39 mediana: 23.45 p95: 61.29. La base de datos solo se consultó una vez por cada cien consultas, porque el resto se contestó con el saldo ya guardado.



Apagué la caché sin tocar el código y repetí la prueba tres veces. Los tiempos fueron para la primera corrida promedio: 117.39 mediana: 60.26 p95: 272.53, para la segunda corrida promedio: 285.13 mediana: 103.17 p95: 1059.14 y para la tercera corrida promedio: 163.20 mediana: 69.87 p95: 621.60.

1. ¿Por qué MongoDB para los avisos y no para el saldo?

Cada aviso tiene datos distintos según el canal. En Mongo cada uno guarda solo lo suyo, y como se escribe una vez y se lee completo, no necesita relaciones. El saldo es dinero y debe ser exacto, con una sola verdad. Eso lo garantiza Postgres, por eso se queda ahí.

2. ¿Qué porcentaje de aciertos medí y qué gané con la caché?

Con cien consultas seguidas fue 99%. Con una transferencia cada diez consultas bajó a 83.3%, porque cada transferencia borra el saldo guardado de Ana y de Beto. Gané sobre todo capacidad, ya que la base de datos queda libre para los cargos y abonos.

3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché?

Podría aprobar un cargo con un saldo viejo. En mi prueba la caché mostraba 295 cuando en la base de datos había 5, así que una transferencia de 100 se habría aprobado sin fondos. Por eso la caché solo muestra el saldo, y el cargo revisa la base de datos, donde respondió “Saldo insuficiente”.

4. ¿Por qué DEL y no SET, y por qué después de guardar y no antes?

Se borra porque es más seguro que actualizar: la siguiente consulta trae el saldo real de la base de datos y no hay dos lugares que calcular. Va después de guardar porque, si fuera antes, otra consulta podría leer el saldo todavía viejo y volver a guardarlo en la caché.

	Con caché	**Sin caché**
Promedio	43.73 ms	188.57 ms
Mediana	25.01 ms	77.77 ms
p95	62.84 ms	651.09 ms
Consultas base de datos	1	100